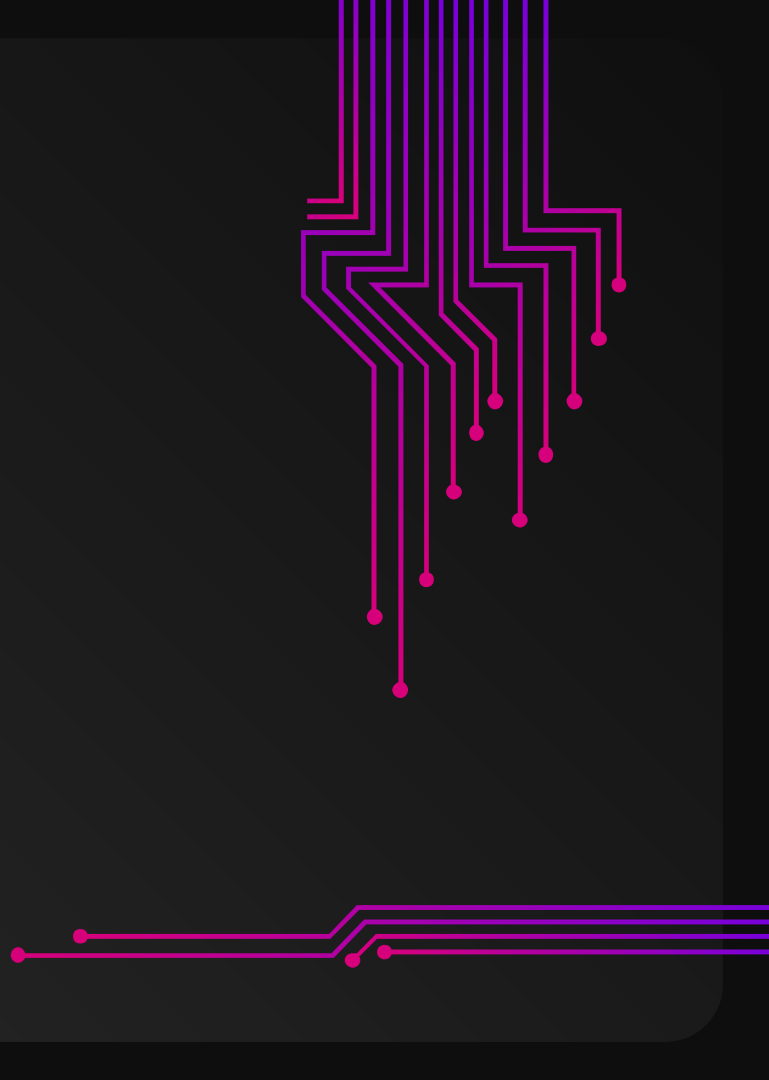
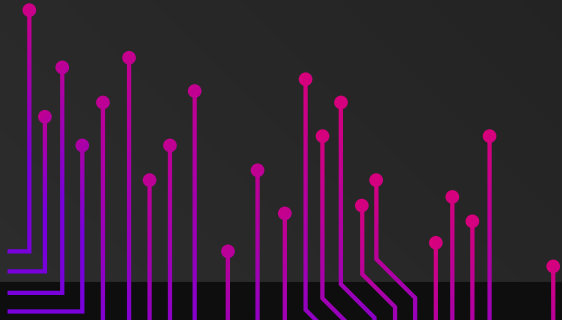


Javascript programming

Prof.Shreya Soni

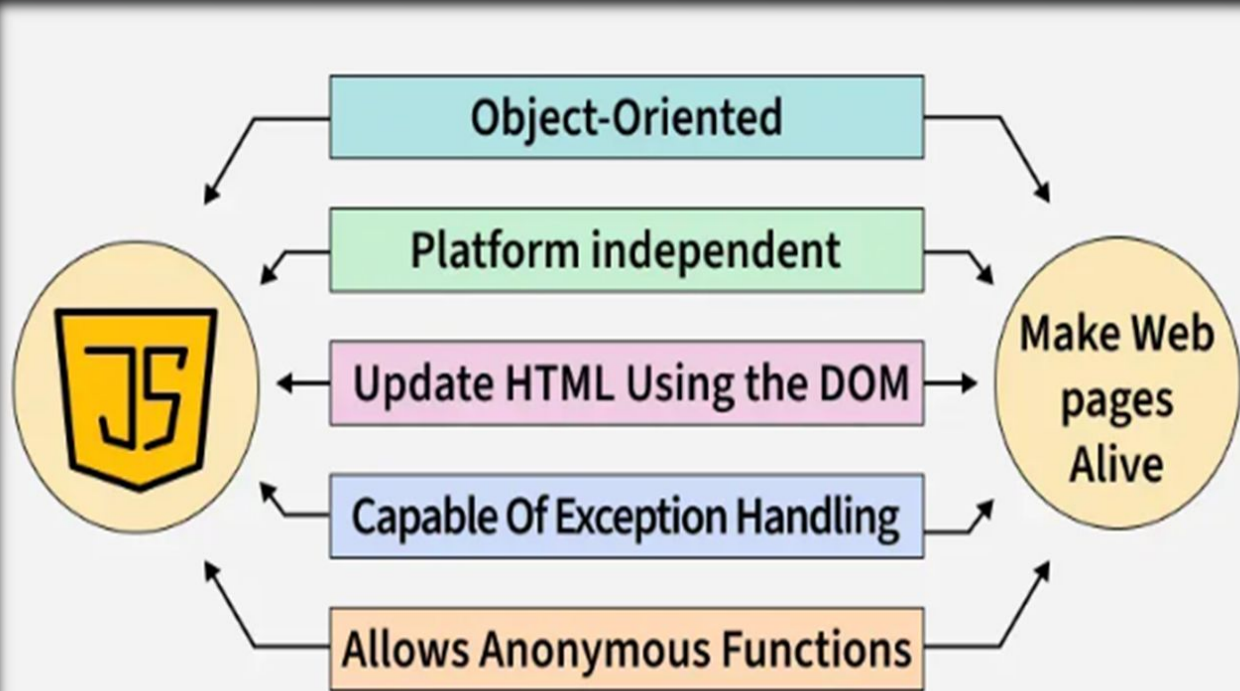


Introduction to Javascript



- ❑ JavaScript is a versatile, dynamically typed programming language that brings life to web pages by making them interactive. It is used for building interactive web applications, supports both client-side and server-side development, and integrates seamlessly with HTML, CSS, and a rich standard library.
 - ❑ JavaScript is a single-threaded language that executes one task at a time.
 - ❑ It is an interpreted language which means it executes the code line by line.
 - ❑ The data type of the variable is decided at run-time in JavaScript, which is why it is called dynamically typed.
- 

Key Features Of Javascript



Why we are using Javascript?

To Make Web Pages Interactive

JavaScript allows users to **interact** with a website.

📌 Examples:

- Button click actions
 - Form validation
 - Dropdown menus
 - Pop-up alerts
-
- ✓ Without JS → web pages are static
 - ✓ With JS → web pages respond to user actions

1. Object-Oriented

JavaScript supports **objects**, **classes**, and **inheritance**, which help you structure code in a reusable and organized way.

Example: You can create objects like car, user, etc., each having properties and methods.

2. Platform Independent

JavaScript runs in any browser and on any operating system (Windows, Mac, Linux, mobile, etc.) without modifications.

You only need a browser or a JS runtime (like Node.js).

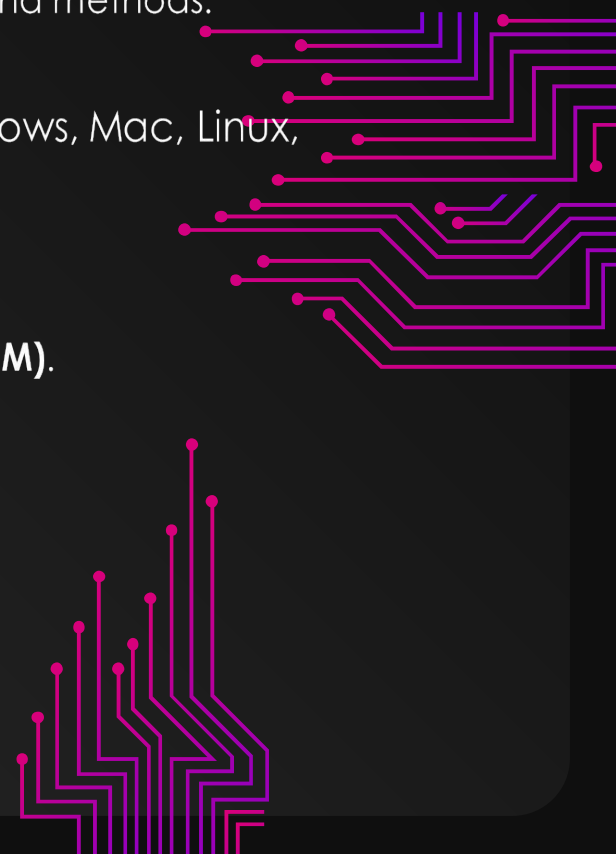
3. Update HTML Using the DOM

JavaScript can interact with and modify the **Document Object Model (DOM)**.

This means it can:

- Change text on the page
- Show/hide elements
- Change styles
- React to user actions

This is a major reason web pages become dynamic.



4. Capable of Exception Handling

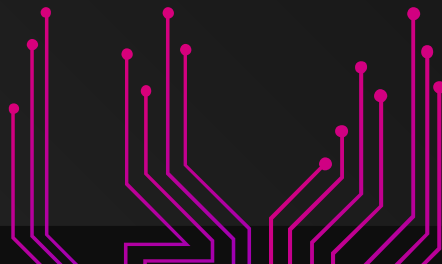
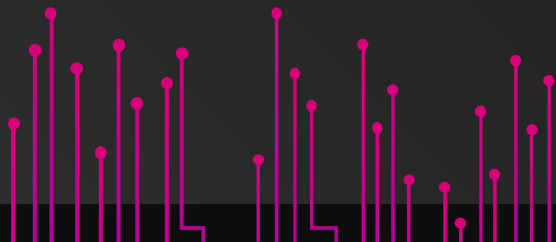
JavaScript supports **try-catch** blocks to handle errors gracefully.

This prevents the whole script from breaking when something goes wrong.

5. Allows Anonymous Functions

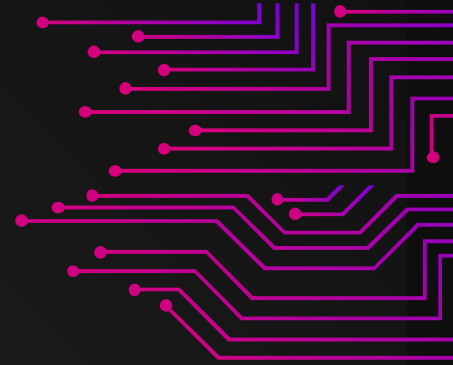
JavaScript can create functions **without names**, often used in:

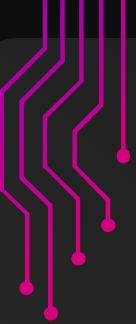
- Callbacks
- Event listeners
- Functional programming
- Array methods (map, filter, etc.)



Advantage of javascript

- 1.Fast execution** – JavaScript runs directly in the user's browser, so it executes quickly without needing server interaction.
- 2.Easy to learn and use** – It has a simple syntax and is beginner-friendly compared to many other programming languages.
- 3.Improves user experience** – Enables interactive features like form validation, animations, pop-ups, and dynamic content.
- 4.Reduces server load** – Many tasks are handled on the client side, decreasing the load on the server.
- 5.Platform independent** – Works on all modern browsers and operating systems.
- 6.Supports event-driven programming** – Responds to user actions such as clicks, typing, and mouse movements.
- 7.Wide range of applications** – Used for web development, mobile apps, games, and server-side development (Node.js).
- 8.Large community and libraries** – Many frameworks and libraries like React, Angular, and Vue make development faster.





1. JavaScript Syntax

JavaScript statements are written line by line and usually end with a semicolon ;.

```
let x = 10; let name = "User";
```

2. JavaScript Comments

Comments are used to explain code and are ignored by the browser.

•Single-line comment

```
// This is a comment
```

•Multi-line comment

```
/* This is a comment */
```

3. JavaScript Data Types

Common basic data types are:

1.Number — 10, 3.5

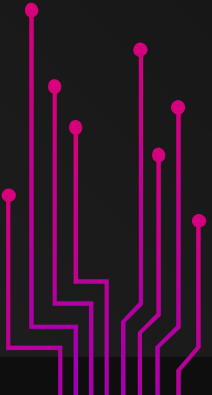
2.String — "Hello"

3.Boolean — true / false

4.Undefined — variable without value

5.Null — empty value

```
let age = 20; // Number let name = "Ram"; // String let isActive = true; // Boolean
```



4. JavaScript Events

Events occur when a user performs an action.

Examples of events:

- onclick
- onmouseover
- onkeydown
- onload

```
<button onclick="alert('Button Clicked')">Click Me</button>
```

JavaScript Variable

A **variable** is used to store data values.

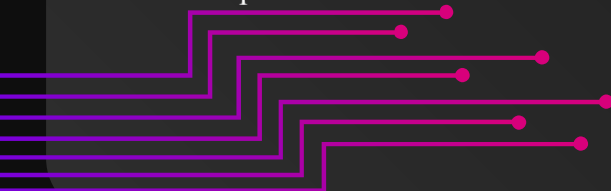
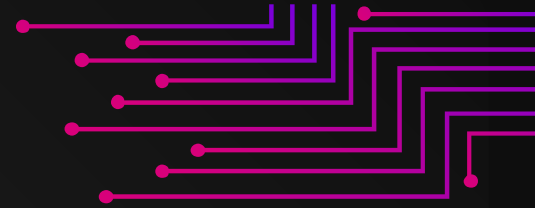
Types of Variables:

- 1.**var** – old method
- 2.**let** – modern and commonly used
- 3.**const** – value cannot be changed

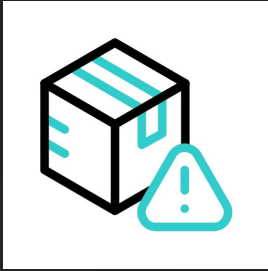
```
var a = 10; let name = "User"; const pi = 3.14;
```

Rules for variables:

- Must start with a letter, `_` or `$`
- Cannot start with a number
- JavaScript is case-sensitive



Javascript has three kinds of popup boxes



Alert box

An alert box is often used if you want to make sure information comes through to the user.

When an alert box pops up, the user will have to click "OK" to proceed.



Confirm box

A confirm box is often used if you want the user to verify or accept something. When a confirm box pops up, the user will have to click either "OK" or "Cancel" to proceed.

If the user clicks "OK", the box returns **true**. If the user clicks "Cancel", the box returns **false**.



Prompt Box

A prompt box is often used if you want the user to input a value before entering a page.

When a prompt box pops up, the user will have to click either "OK" or "Cancel" to proceed after entering an input value.

If the user clicks "OK" the box returns the input value. If the user clicks "Cancel" the box returns null.

What is The DOM?

The DOM is a W3C (World Wide Web Consortium) standard.

The DOM defines a standard for accessing documents:

”The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document.”

Properties of the DOM

Node based

Everything in the DOM is a "node," like elements (tags), text, and attributes.

Tree structure

The DOM is like a family tree with parents, children, and siblings.

Live Updates

If you change something in the DOM with JavaScript, it instantly updates on the page

Works everywhere

The DOM works on all browsers, platforms, and programming languages.

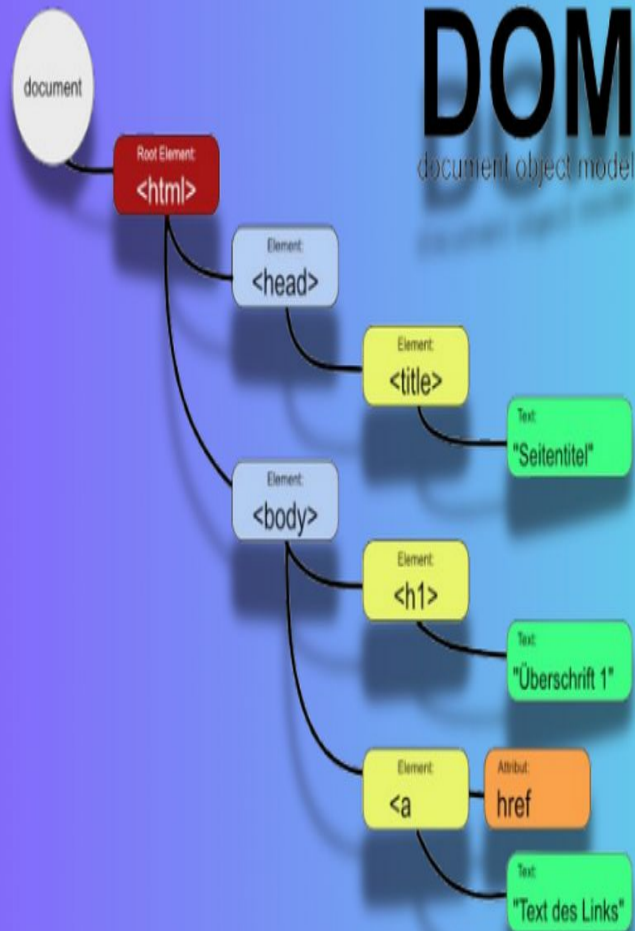
How it's work

- Access elements (like finding an `<h1>` tag).
- Modify content (like changing the text of a `<p>` tag).
- React to events (like a button click).
- Create or remove elements dynamically

•

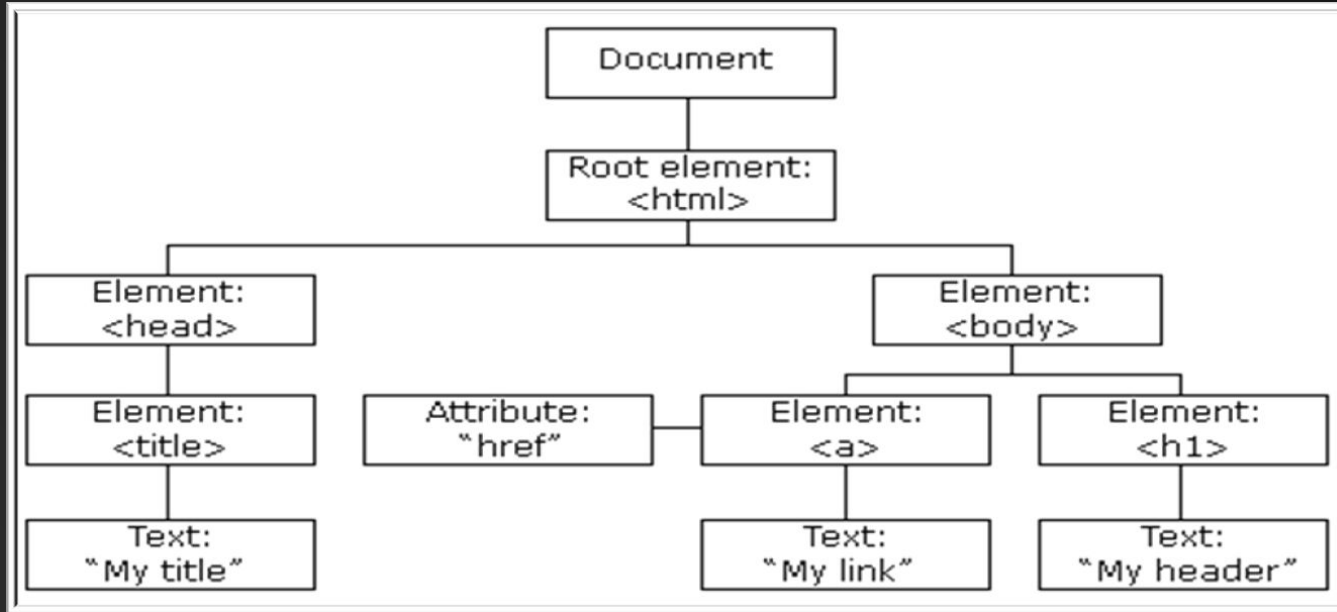
What is HTML DOM?

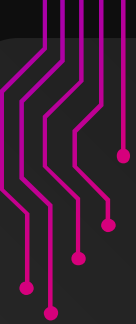
- ❖ The HTML DOM is a standard object model and programming interface for HTML.
- ❖ The HTML elements as objects
- ❖ The properties of all HTML elements
- ❖ The methods to access all HTML elements
- ❖ The events for all HTML elements
- ❖ In other words: The HTML DOM is a standard for how to get, change, add, or delete HTML elements.



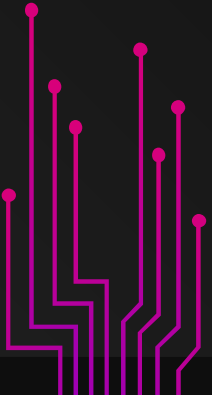
Javascript HTML DOM

- When a web page is loaded, the browser creates a **Document Object Model** of the page.
- With the HTML DOM, JavaScript can access and change all the elements of an HTML document.
- The **HTML DOM** model is constructed as a tree of **Objects**:





With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page
 - JavaScript can change all the HTML attributes in the page
 - JavaScript can change all the CSS styles in the page
 - JavaScript can remove existing HTML elements and attributes
 - JavaScript can add new HTML elements and attributes
 - JavaScript can react to all existing HTML events in the page
 - JavaScript can create new HTML events in the page
- 



Methods

Description

getElementById(id)

Selects an element by its ID.

getElementsByClassName(class)

Selects all elements with a given class.

querySelector(selector)

Selects the first matching element.

querySelectorAll(selector)

Selects all matching elements.

createElement(tag)

Creates a new HTML element.

appendChild(node)

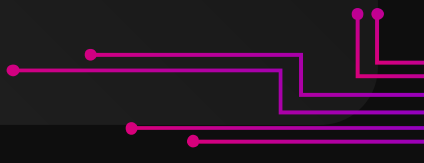
Adds a child node to an element.

remove()

Removes an element from the DOM.

addEventListener(event, fn)

Attaches an event handler to an element.





```
<div className="dropdown col-md-12">
  {/ level 1 */}
  <CSSTransition
    in={activeMenu === "main"}
    timeout={500}
    classNames="menu-primary"
    unmountOnExit
    onEnter={calcHeight}
  >
    <div className="menu">
      {categoryList.map(val => {
        return (
          <DropdownItem
            leftIcon={val.icon}
            goToMenu={val.id}
            key={val.id}
            rightIcon={<RightArrowIcon />}
          >
            <h4>{val.name}</h4>
          </DropdownItem>
        );
      })}
      {linkList.map(val => {
        return (
          <DropdownItem
            goToMenu="main"
            leftIcon={<BoltIcon />}
            key={val.id}
            goHref={val.link}
          >
            {val.name}
          </DropdownItem>
        );
      })}
    </div>
  </CSSTransition>
{/ level 2 */}
<CSSTransition
  in={activeMenu === linkName}
  timeout={500}
  classNames="menu-secondary"
  unmountOnExit
  onEnter={calcHeight}
  >
    <div className="menu">
      <DropdownItem goToMenu="main" leftIcon={<ArrowIcon />}>
        <h4>Kembali</h4>
      </DropdownItem>
      {linkList.map(valLink => {
        return (
          <DropdownItem
            leftIcon={<BoltIcon />}
            goToMenu={linkName}
            goHref={valLink.link}
          >
            {valLink.name}
          </DropdownItem>
        );
      })}
    </div>
  </CSSTransition>
</div>
```

```
"eslintConfig": {
  "extends": [
    "react-app",
    "react-app/"
  ],
},
{
  position: fixed;
  left: 0;
  top: 0;
  right: 0;
  bottom: 0;
  background-color: rgba(14, 7, 7, 0.514);
  opacity: 0;
  transition: all 0.3s ease-in-out;
  pointer-events: none;
}
display: flex;
align-items: start;
justify-content: center;
}
.App {
  text-align: center;
}
```

JAVASCRIPT PROGRAMMING

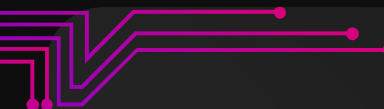
Javascript RegExp(Regular expression)

- A regular expression is a special sequence of characters that defines a search pattern, typically used for pattern matching within text. It's often used for tasks such as validating email addresses, phone numbers, or checking if a string contains certain patterns (like dates, specific words, etc.).
- In JavaScript, RegExp is an object that is used to create regular expressions, which are patterns used to match character combinations in strings



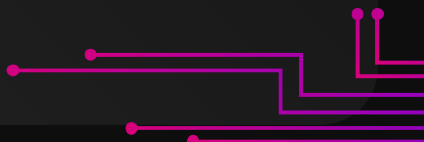
Javascript RegExp(Regular expression)

Symbol	Meaning
<code>^</code>	Start of string
<code>\w+</code>	One or more letters/numbers
<code>@</code>	Must contain @
<code>.</code>	Dot (.)
<code>\$</code>	End of string



Using String Methods

Method	Description
<code>match(regex)</code>	Returns an Array of results
<code>replace(regex)</code>	Returns a new String
<code>search(regex)</code>	Returns the index of the first match

- In javascript, regular expression are often used with the two **string methods**: `search()` and `replace()`.
 - **The `search()` method** uses an expression to search for a match, and returns the position match.
 - **The `replace()` method** returns a modified string where the pattern is replaced.
- 

Examples of Javascript:

```
<!DOCTYPE html>
<html>
<body>

<h2>What Can JavaScript Do?</h2>

<p>JavaScript can change HTML attribute values.</p>

<p>In this case JavaScript changes the value of the src (source) attribute of an image.</p>

<button onclick="document.getElementById('myImage').src='pic_bulbon.gif'">Turn on the
light</button>



<button onclick="document.getElementById('myImage').src='pic_bulboff.gif'">Turn off the
light</button>

</body>
</html>
```



How **innerHTML** Works

✓ 1. Reading the content

```
document.getElementById("demo").innerHTML;
```

This returns whatever is written inside the element with id **"demo"**.

Examples of Javascript:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>My First JavaScript</h1>
```

```
<button type="button" onclick="document.getElementById('demo'). Date()">
```

```
Click me to display Date and Time.</button>
```

```
<p id="demo"></p>
```

```
</body>
```

```
</html>
```


Examples of Javascript:

Example 1:

```
let sentence = "Hello World";  
let result = sentence.match(/Hello/);  
  
console.log(result);
```

Example 2:

```
<script>  
let str = "Hello, world!";  
let index = str.search("world");  
console.log(index);  
</script>
```

Example 3:

```
<script>  
let text = "I like apples";  
let newText = text.replace("apples", "bananas");  
console.log(newText);  
</script>
```

Example 4:

```
<script>  
let text = "I love apples. Apples are my favorite  
fruit. I eat apples every day.";  
let newText = text.replaceAll("apples", "bananas");  
  
console.log(newText);  
</script>
```

What is form validation?

- Form validation ensures that the data entered into a web form is accurate, complete, and meets the specified requirements before being sent to the server.
- **Form Validation** means checking the user's input in a form to make sure the data is **correct, complete, and in the proper format** before submitting it.

What is form validation?

Types of validation



Client side

Client-side validation is the validation that is done **inside the user's browser** using **JavaScript or HTML** before the form is sent to the server.



Server side

Server-side validation is the validation that is done on the server after the form data is sent from the browser.

Examples of Javascript:

Symbol	Meaning
^	Start of string
\w+	One or more letters/numbers
@	Must contain @
.	Dot (.)
\$	End of string

```
<form onsubmit="return validate()">
  <input type="text" id="email">
  <input type="submit">
</form>
```

```
<script>
function validate() {
  var email = document.getElementById("email").value;
  var pattern = /^\\w+@\\w+\\.\\w+$/;

  if (!pattern.test(email)) {
    alert("Invalid Email");
    return false;
  }
  alert("Valid Email");
  return true;
}
</script>
```

this keyword

The `this` keyword in JavaScript is a special keyword that refers to the current object in which the code is being executed. Its value is determined at runtime and depends on how and where a function is called, not where it is written.

Definition

In JavaScript, the `this` keyword refers to the object that is currently invoking (calling) a function.

Uses of this Keyword

- To access **object properties and methods**
- To refer to the **current object**
- To make code **reusable** for different objects
- Commonly used in **objects, events, and classes**

this keyword

Behavior of `this` Keyword

1. Global Context

- In the global scope (in a browser), `this` refers to the **window object**.

2. Inside an Object Method

- When used inside a method of an object, `this` refers to the **object itself**.

3. Inside a Function

- In a regular function, `this` refers to the **global object** (or `undefined` in strict mode).

4. In Event Handling

- In event handlers, `this` refers to the **HTML element** that triggered the event.

5. In Arrow Functions

- Arrow functions do **not have their own** `this`.

They inherit `this` from their **surrounding scope**

this keyword

Advantages of Using `this`

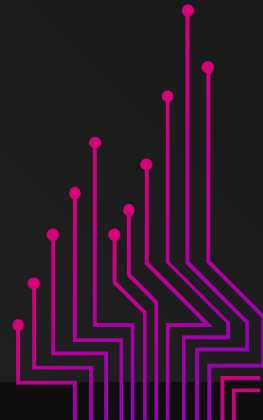
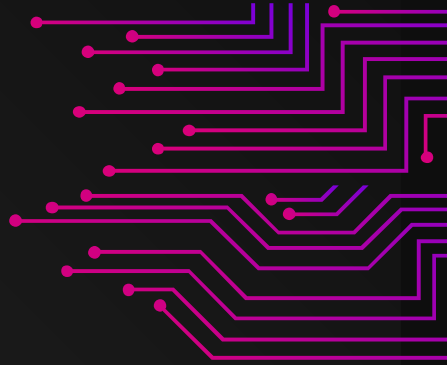
- Helps avoid repetition of object names
- Makes code cleaner and more readable
- Supports object-oriented programming concepts

Javascript Function

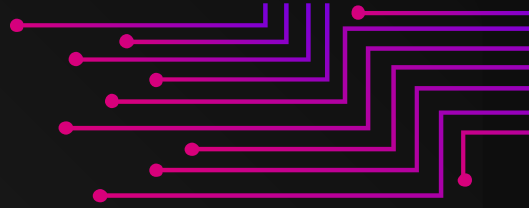
- A **function in JavaScript** is a block of reusable code designed to perform a specific task.
- Functions help in **code reusability**, **modularity**, and **easy maintenance** of programs.
- A JavaScript function is a set of statements that performs a task or calculates a value and executes when it is called
- Functions are executed when they are "called" or "invoked".

Syntax

```
function functionName(parameters)
{
    // code to be executed
}
```



Javascript Function



Functions are defined with the `function` keyword:

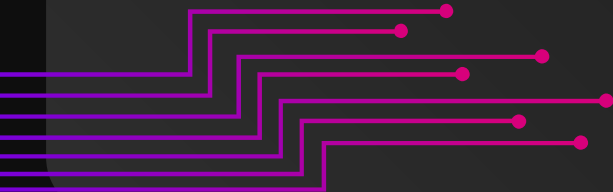
- followed by the function name
- followed by parentheses ()
- followed by brackets { }

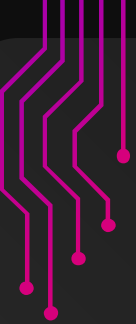
The function name follows the naming rules for variables.

Optional parameters are listed inside parentheses: (*p1*, *p2*, ...)

Code to be executed is listed inside curly brackets: { }

Functions can return an optional value back to the caller.





Javascript Function

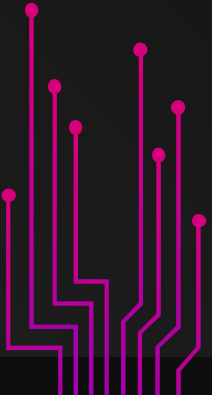
Why Functions?

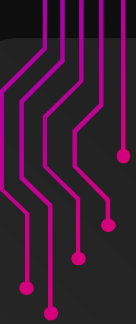
Functions enable better code organization and efficiency.

With functions you can reuse code.

You can write code that can be used many times.

You can use the same code with different arguments, to produce different results





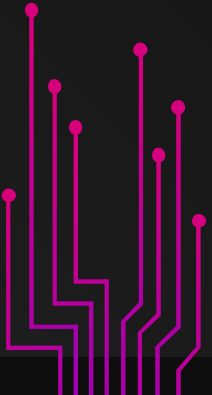
Javascript Function

Function Invocation ()

The code inside the function will execute when "something" invokes (calls) the function:

- When it is invoked (called) from JavaScript code
- When an event occurs (a user clicks a button)
- Automatically (self invoked)

The () operator invokes a the function.



Example Of Javascript Function

```
<!DOCTYPE html>
<html>
<body>

<h2>Addition using Function</h2>
<p id="add"></p>

<script>
function add(a, b) {
    return a + b;
}

let result = add(10, 20);
document.getElementById("add").innerHTML = result;
</script>

</body>
</html>
```

Example Of Javascript Function

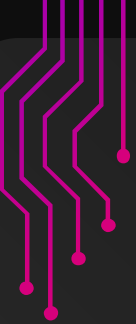
```
<!DOCTYPE html>
<html>
<body>

<h2>Even or Odd</h2>
<p id="evenodd"></p>

<script>
function checkNumber (n) {
    if (n % 2 == 0) {
        return "Even";
    } else {
        return "Odd";
    }
}

let ans = checkNumber (16);
document.getElementById ("evenodd").innerHTML = ans;
</script>

</body>
</html>
```



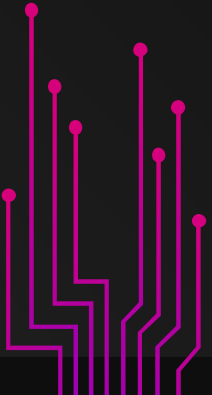
Javascript Function

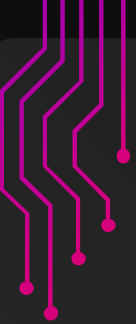
Function Invocation ()

The code inside the function will execute when "something" invokes (calls) the function:

- When it is invoked (called) from JavaScript code
- When an event occurs (a user clicks a button)
- Automatically (self invoked)

The () operator invokes a the function.





Javascript Function Parameters

A JavaScript `function` does not perform any checking on parameter values (arguments).

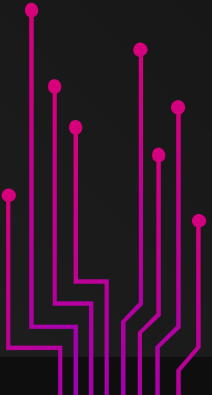
Function Parameters and Arguments

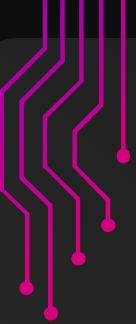
Earlier in this tutorial, you learned that functions can have parameters:

```
function functionName(parameter1, parameter2, parameter3) {  
  
    // code to be executed  
  
}
```

Function parameters are the names listed in the function definition.

Function arguments are the real values passed to (and received by) the function.





Parameter Rules

JavaScript function definitions do not specify data types for parameters.

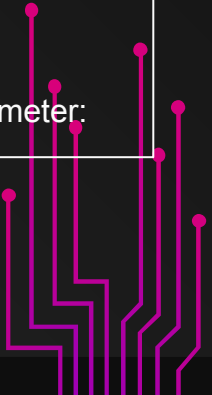
JavaScript functions do not perform type checking on the passed arguments.

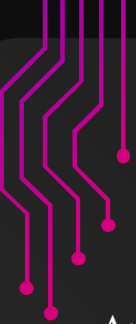
JavaScript functions do not check the number of arguments received.

Default Parameters

If a function is called with missing arguments (less than declared), the missing values are set to `undefined`.

Sometimes this is acceptable, but sometimes it is better to assign a default value to the parameter:

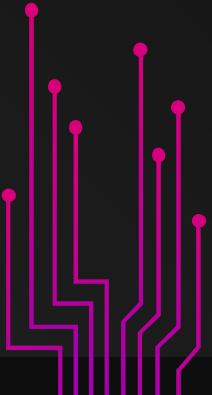




Javascript Arrow function:

An Arrow function: An arrow function in JavaScript is a shorter and more concise way to write functions.

It uses the `=>` syntax. Arrow functions are often used for their simplicity and to avoid issues with the `this` keyword, as they do not bind their own `this` value; instead, they inherit it from the surrounding scope.



Example Of Javascript Function

```
<!DOCTYPE html>

<html>

<body>

<h1>JavaScript Functions</h1>

<p>This example shows the syntax of a function, without the use of arrow function syntax.</p>

<p id="demo"></p>

<script>
let hello = function() {
    return "Hello World!";
}

document.getElementById("demo").innerHTML = hello();
</script>

</body>
</html>
```

Example Of Javascript Function

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Functions</h1>
<h2>The Arrow Function</h2>

<p>This example shows the syntax of an Arrow Function, and how to use it.</p>

<p id="demo"></p>

<script>
// myFunction computes the product of a and b
myFunction = (a, b) => a * b;

let result = myFunction(4, 5);
document.getElementById("demo").innerHTML = "The product is: " + result
</script>

</body>
</html>
```